Project Definition & Development Contract

Building a Vulkan Renderer in C++20

De Montfort University | BSc (Hons) Games Production

Mohamed Deeq Mohamed | P2884884 | 28th March, 2026

Version 11.0: Vulkan 1.3 Dynamic Rendering; vk-bootstrap throughout; MoSCoW prioritisation; no `VkRenderPass`, no `VkFramebuffer`

1. Student Name:	Mohamed Deeq Mohamed	**2. P-Number:**	P2884884
3. Project Title:	Building a Vulkan Renderer in C++20	**2a. Deadline:**	1 June 2026
4. Supervisor(s):	Salim Hashu / Dr Conor Fahy	**2b. Start:**	30 March 2026

This document serves as both the Project Definition and a formal Development Contract. The objectives stated here constitute a binding contract; the final deliverable will be evaluated, in part, against them.

5. Introduction (max. 100 words)

I am a final-year Games Production student at De Montfort University with a strong interest in low-level graphics programming and emerging rendering research. This project involves building a real-time 3D renderer from scratch using the Vulkan 1.3 graphics API and C++20. Vulkan initialisation is handled through **vk-bootstrap**, while all rendering code is written directly against the Vulkan API using **Dynamic Rendering**, eliminating `VkRenderPass` and `VkFramebuffer` entirely. The project serves both as an undergraduate assessment and as a deliberate pilot study for a planned PhD in neural rendering at De Montfort University beginning in October 2026, providing codebase foundations intended to be built upon after graduation.

6. Aims (max. 100 words)

This project aims to develop a thorough, working understanding of how modern GPUs process and render geometry by implementing a Vulkan 1.3 renderer in C++20. Rather than following a tutorial, I intend to make genuine design decisions and understand their consequences, from synchronisation barriers to descriptor sets to physically based shading mathematics. Vulkan initialisation boilerplate is abstracted through vk-bootstrap, consistent with professional practice. A secondary aim is to produce a clean, extensible codebase that remains useful after graduation, serving as a foundation for PhD research in neural rendering, Gaussian splatting, and GPU-accelerated scene reconstruction.

6a. State of the Art

Modern real-time graphics is built upon physically based rendering (PBR) models that approximate light transport using microfacet BRDFs, such as the Cook–Torrance formulation and the Disney principled BRDF used in feature film production [1]. These models link surface roughness, metalness, and view-dependent specular response to physically interpretable parameters and have become the de facto shading basis for both offline renderers and game engines.

Vulkan 1.3's dynamic rendering path, which removes the need for explicit render pass and framebuffer objects [2], is now recommended as the modern baseline for new engines. It simplifies render pass management while retaining full low-level control over the pipeline.

In parallel, neural rendering has emerged as a significant research direction, representing scenes as neural radiance fields (NeRF) that are optimised from multi-view images to enable high-quality novel view synthesis [3]. Surveys on neural rendering highlight ongoing trade-offs between image quality, training time, and real-time inference performance [4].

Most recently, 3D Gaussian Splatting (3DGS) has been proposed as an explicit, GPU-friendly alternative to volumetric NeRFs. In this approach, scenes are represented as millions of anisotropic 3D Gaussian primitives that are rasterised efficiently to achieve real-time radiance field rendering at 1080p while maintaining state-of-the-art reconstruction quality [5]. Follow-up surveys on Gaussian splatting and neural rendering emphasise the need for robust, extensible rendering back-ends capable of mixing classical rasterisation, PBR shading, and neural scene representations [6]. This project positions a Vulkan 1.3 renderer as precisely such a back-end, with Gaussian splat rendering serving as an initial bridge towards future PhD research in neural rendering and scene reconstruction.

7. Objectives (max. 200 words)

Each objective is colour-coded by MoSCoW priority (see Appendix B): Must Have Should Have Could Have

1. **Baseline Vulkan Pipeline (Week 2), *Must Have*:** Set up a Vulkan 1.3 environment via vk-bootstrap and render a coloured triangle using Dynamic Rendering. Neither `VkRenderPass` nor `VkFramebuffer` shall appear anywhere in the project.
2. **Rotating 3D Cube (Week 3), *Must Have*:** Implement vertex and index buffers via staging buffer upload, attach a VMA-managed depth image dynamically, and supply MVP transformation data through a UBO and descriptor set.
3. **OBJ Loading and Texture Mapping (Week 5), *Must Have*:** Load a mesh via `tinyobjloader`, apply a texture through a combined image sampler, use `synchronization2` for layout transitions, and generate a mipmap chain via `vkCmdBlitImage`.
4. **Technical Report and Renderer Polish (Week 9), *Must Have*:** Deliver an interactive camera, swapchain resize handling, zero validation errors, a reproducible CMake build, and a complete eight-section technical report including at least one RenderDoc-based performance analysis figure.

5. **PBR Shading (Week 7), *Could Have*:** Implement the Cook–Torrance BRDF [7] (GGX normal distribution, Smith geometry term, Schlick Fresnel approximation) with Dear ImGui runtime controls. The minimum viable implementation is Lambertian diffuse shading.
6. **Gaussian Splatting (Post Week 7), *Could Have*:** Render a `.ply` Gaussian splat scene as a PhD research pilot study, following the method described by Kerbl et al. [5].
7. **XR Exploration (If time permits), *Could Have*:** Begin OpenXR integration or document it as a direction for further work.

8. Deliverables (max. 100 words)

- A real-time Vulkan 1.3 renderer written in C++20 using Dynamic Rendering, featuring OBJ loading, texture mapping with mipmapping, and Dear ImGui runtime controls. PBR shading and Gaussian splat rendering (`.ply` input) are pursued as Could Have extensions.
- An existing public GitHub repository created for this project, maintaining a sustained commit history and reproducible via `CMakePresets.json` on both Linux and Windows using vcpkg.

- An eight-section technical report covering architecture, pipeline design, PBR mathematics, RenderDoc performance data, and critical reflection.
- Per-milestone evidence packages consisting of annotated screenshots and validation layer logs.

9. Schedule of Activities

Week	Focus	Key Tasks
1	Setup and toolchain	Configure CMake with vcpkg (vk-bootstrap); install the Vulkan SDK, validation layers, and RenderDoc; initialise the GitHub repository.
2	Triangle render	Complete vk-bootstrap initialisation; build the SPIR-V pipeline; implement the Dynamic Rendering command sequence; write the layout transition helper.
3	Rotating cube	Create vertex and index buffers; implement staging buffer uploads; set up the VMA depth image; configure UBO and descriptor sets; apply MVP transforms.
4	Consolidation and report	Conduct an RAII audit; begin writing report sections 1 and 2 (Introduction, Architecture).
5	Textured OBJ model	Integrate `tinyobjloader`; set up the combined image sampler; apply `synchronization2` barriers; generate the mipmap chain.
6	Report (parallel)	Write report sections 3 and 4 (Pipeline, Shading Model); conduct BRDF research.
7	PBR shading *(Could Have)*	Implement GGX NDF, Smith G, and Schlick Fresnel; build the Dear ImGui material panel; create Blender reference comparisons.
7–8	Gaussian splatting *(Could Have)*	Parse `.ply` files; allocate GPU splat buffers; implement depth sorting, ellipse projection, and alpha compositing.
8	Renderer polish	Implement the camera; handle swapchain resizing; achieve zero validation errors; perform RenderDoc frame captures.
9	Final submission	Complete report sections 5 through 8; tidy the GitHub repository; verify the CMake build; obtain supervisor sign-off.

10. Requirements

Software: Vulkan SDK 1.3, C++20 (GCC/MSVC/Clang), CMake 3.25+, vcpkg, vk-bootstrap, Vulkan Memory Allocator, tinyobjloader, tinyply, Dear ImGui, GLM, stb_image, spdlog, shaderc/glslc, RenderDoc, JetBrains CLion, Visual Studio 2022, GitHub, and Blender (Cycles).

Hardware: A GPU with Vulkan 1.3 support (AMD or NVIDIA discrete). Windows 10/11 or Linux (Arch). Access to a VR headset is required only if the XR stretch goal is reached and is not a hard requirement.

11. Research Hypothesis

> *"Can building a Vulkan rendering pipeline from scratch, using industry-representative abstractions alongside direct API usage, establish the GPU architecture understanding and codebase foundations needed for postgraduate research in neural rendering and AI-driven graphics?"*

This hypothesis is tested by designing the renderer to be extensible beyond the assessed milestones: towards Gaussian splatting (a direct precursor to neural rendering [5]), ray tracing, and XR. The critical reflection section of the technical report will evaluate whether this approach

has succeeded.

Supervisory Agreement & Signatures

Party	Signature	Date
12. Student: Mohamed Deeq Mohamed (P2884884)	_____	_____
13. Supervisor: Salim Hashu	_____	_____
Supervisor: Dr Conor Fahy	_____	_____

14. Date: _____

Appendix A: Grade Band Mapping & Milestone Contract

The table below defines the minimum milestone completion required for each grade band. **Could Have milestones (PBR, Gaussian splatting, XR) are not required for any grade band.** Failing to attempt them will not penalise the assessment outcome.

Grade	Mark	Minimum	Rationale
Fail	<40%	Below Obj. 1	No functional Vulkan output.
Third	40–49%	Objectives 1–2 + Report §1–3	Working pipeline with triangle and rotating cube.
2:2	50–59%	Objectives 1–3 + Report §1–4	Textured OBJ model with full texture pipeline.
2:1	60–69%	Objectives 1–3 + Report §1–6 + polish	Camera, resize handling, RenderDoc analysis, and a strong report.
First	**70%+**	**Objectives 1–4 + full report**	**All eight report sections; high code quality; critical reflection. PBR and stretch goals are not required.**

Important: A First Class Honours outcome is achievable through Objectives 1 to 4 (baseline pipeline through textured OBJ, renderer polish, and the full technical report) without any PBR or stretch goal implementation. Could Have objectives are pursued for personal and research benefit only; they are not assessed differentiators.

#	Milestone	Priority	Grade Band	Target	Sign-off
1	Baseline Vulkan pipeline + triangle	Must Have	Third (40%+)	Week 2	_____
2	Rotating 3D cube	Must Have	2:2 (50%+)	Week 3	_____
3	OBJ loading + texture mapping	Must Have	2:1+ (60%+)	Week 5	_____
4	Renderer polish + full report	Must Have	First (70%+)	Week 9	_____
5	PBR shading	Could Have	Not graded	Week 7	_____
6	Gaussian splatting (`.ply`)	Could Have	Not graded	Post Obj. 3	_____
7	XR/VR exploration	Could Have	Not graded	If time	_____

Appendix B: MoSCoW Prioritisation

Must Have

- Vulkan 1.3 Dynamic Rendering throughout. Neither `VkRenderPass` nor `VkFramebuffer` shall appear anywhere in the codebase.
- Baseline triangle render via `vkCmdBeginRendering` (Objective 1).
- Rotating 3D cube with VMA depth image, UBO, and MVP transforms (Objective 2).
- Textured OBJ model with mipmapping (Objective 3).
- RAII-compliant C++20 codebase with zero validation errors on both startup and shutdown.
- Reproducible CMake build on Linux and Windows via vcpkg.
- Public GitHub repository with a sustained commit history.
- Full eight-section technical report (Objective 4).

Should Have

- Interactive orbit or first-person camera.
- Swapchain resize handling (`VK_ERROR_OUT_OF_DATE_KHR`).
- Dear ImGui overlay.
- RenderDoc frame capture analysis with GPU timing data.
- Per-milestone evidence packages (screenshots, validation logs, written notes).

Could Have (Not Required for Any Grade Band)

- **PBR shading:** Cook–Torrance BRDF [7] (GGX normal distribution, Smith geometry term, Schlick Fresnel approximation) with Dear ImGui material controls. The minimum viable implementation is Lambertian diffuse. This objective is pursued because understanding shading mathematics is directly relevant to PhD research in neural rendering.
- **Gaussian splat rendering:** Ingest a `.ply` file, sort and project 3D Gaussians to screen-space ellipses, and alpha-composite the result, following the method described

by Kerbl et al. [5]. This objective constitutes a direct pilot for PhD research in Gaussian-based neural rendering at De Montfort University and will only be started once Objectives 1 through 3 have been signed off.

- **VR/AR/XR exploration via OpenXR:** Investigate integrating OpenXR into the renderer. Vulkan is the natural XR backend and Dynamic Rendering is compatible with per-eye render structures. A full implementation is unlikely within the project timeline; however, even partial progress or a detailed further-work discussion in the report would be valuable. This objective is pursued only after Gaussian splatting is complete or formally deferred.

Won't Have (Explicitly Out of Scope)

This is a **renderer**, not a game engine. The following features will not be implemented: physics simulation, entity-component systems, scene graphs, scripting runtimes, audio, level editor tooling, networking, shadow mapping, deferred shading, a full ray tracing pipeline, mobile or console Vulkan, NeRF training or inference, and multi-GPU rendering.

Appendix C: Risk Register

Risk	Likelihood	Impact	Mitigation
Report deprioritised in favour of coding	High	High	Writing begins in Week 4. Section 5 (Challenges) is maintained as a daily log from Day 1 and is never written retrospectively.
Vulkan synchronisation errors consuming time	High	Medium	Validation layers remain enabled at all times. `synchronization2` is used for all barriers. RenderDoc is employed from Milestone 1 onwards.
Gaussian splatting scope creep displacing core polish	Medium	High	Objective 6 is strictly not started until Objectives 1 through 3 have been signed off. Any time pressure on Objective 4 immediately suspends all stretch work.
Image layout transition errors under Dynamic Rendering	Medium	Medium	A reusable `transitionImageLayout` helper is written at Objective 1 and applied consistently thereafter.
vk-bootstrap incompatibility on DMU lab GPU drivers	Low	Medium	Full fallback control is available via vk-bootstrap selectors. The `uni-debug` preset is verified on a lab machine before Objective 1 sign-off.
CMake/vcpkg build failures across platforms	Low	Medium	Both presets are verified on clean environments before submission.

Key: High Medium Low

Appendix D: Student Commitments

- Milestone evidence is presented at each weekly check-in.
- Report writing begins no later than **Week 4**. Section 5 (Challenges) is maintained as a running log from Day 1.
- The vk-bootstrap initialisation sequence is documented in Report Sections 2 and 7 to demonstrate understanding regardless of the abstraction used.
- **Objective 6 (Gaussian splatting) is not started until Objectives 1 through 3 have been signed off.** Time pressure on Objective 4 immediately pauses all stretch work.

- Scope additions are discussed with supervisors before implementation.
- Work is tracked in the existing public GitHub repository with regular, descriptive commits and the use of issues where appropriate.
- Project management is conducted through a dedicated Jira Software project configured with epics mirroring Objectives 1 through 7 to keep progress visible and accountable.

Supervisory responsibilities: weekly milestone review (approximately 30 minutes); sign-off on evidence packages; written report feedback no later than one week before the submission deadline.

References

[1] B. Burley, "Physically based shading at Disney," *SIGGRAPH Course: Practical Physically Based Shading in Film and Game Production*, 2012.

[2] Khronos Group, "Vulkan guide: Dynamic rendering," https://docs.vulkan.org/guide/latest/dynamic_rendering.html, 2023, accessed: Mar. 2026.

[3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2022.

[4] A. Tewari, J. Thies, B. Mildenhall, P. Srinivasan, E. Tretschk, W. Yifan, C. Lassner, V. Sitzmann, R. Martin-Brualla, S. Lombardi, T. Simon, C. Theobalt, M. Nießner, J. T. Barron, G. Wetzstein, M. Zollhöfer, and V. Golyanik, "Advances in neural rendering," *Computer Graphics Forum*, vol. 41, no. 2, pp. 703–735, 2022.

[5] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

[6] Y. Chen, G. Chen, C. Wang *et al.*, "A survey on 3D Gaussian splatting," *arXiv preprint arXiv:2401.03890*, 2024.

[7] R. L. Cook and K. E. Torrance, "A reflectance model for computer graphics," *ACM Transactions on Graphics*, vol. 1, no. 1, pp. 7–24, 1982.